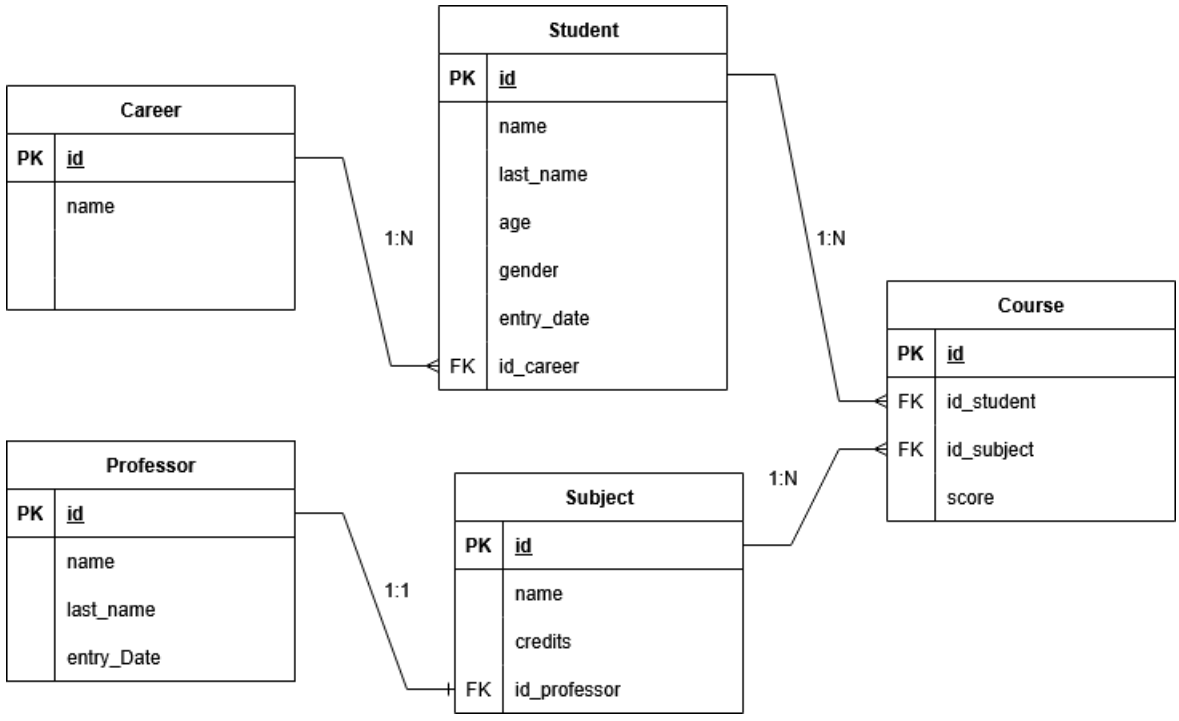
REST API for managing student and professors registrations at a college.

-
- A college needs an API that facilitates basic registration for students, careers, professors, and subjects:
-
- A student can register for multiple subjects.
- A subject can have multiple students.
- A student belongs to only one career.
- A professor can only teach one subject.

- This project aims to demonstrate:
  • The use of Spring Boot with JPA for database management.
  • The use of a MySQL relational database.
  • Exposing RESTful endpoints to perform CRUD operations (GET, POST, PUT, DELETE, or other methods deemed necessary).
  • The use of DTOs to separate domain models from external representation.
  • Proper error handling with ResponseEntity, correct HTTP status codes, and clear messages.
  • The use of lambdas or streams.
  • The use of Enums and Exceptions.
  • Modular organization (Service, Repository, Controller).
  • The use of Docker.
-
- **Solution:**
- To meet the requirements, a total of 5 entities are defined: Career, Student, Professor, Subject, and Course. These represent careers, students, professors, subjects, and courses, respectively, with Course being the entity that facilitates the many-to-many relationship between subjects and students.
-



- **Data Management**
- The image above specifies the entities, their attributes, and the relationships between them. Data integrity is defined as follows:
- • When registering an item in the database, the system provides the record ID. Foreign keys are mandatory, and a foreign key combination can only appear once; for example, in the Course entity, a student cannot register twice for the same course.
- • Data deletion will only be effective if the primary key (PK) of a record is not linked to the foreign key (FK) of another record; this is to maintain data integrity, so the CASCADE function does not apply to this API.
- • Updating records is permitted for all fields except for the primary key of entities and the foreign keys of the Course entity.

RESTful Endpoints

- For this project, the server will be localhost and its port 8080; therefore, the URL will look like this:
- http://localhost:8080/api/{endPoint}
- Where the endpoint can be /careers, /students, /professors, /subjects, or /courses.
- **GET and POST Services**
- http://localhost:8080/api/{endPoint}: The GET service will retrieve all records of an entity from the database.
- http://localhost:8080/api/{endPoint}: The POST service will save only one record of an entity in the database.
- **PUT and DELETE Services**
- http://localhost:8080/api/{endPoint}/{id}: The PUT service requires the record ID as a parameter; if it exists, the data will be updated. Otherwise, a 400 response will be returned.
- http://localhost:8080/api/{endPoint}/{id}: The DELETE service requires the record ID as a parameter; if it exists, the data will be updated; otherwise, a 400 response will be returned.